PARCEL

Property visualization & floor-plan intake

Software design and implementation handoff
Version 0.1 / 4 October 2026

A working prototype with explicit integration boundaries

The prototype separates original extraction, human review, publication and evaluation. This document identifies what is implemented, how to run it, and what remains unconnected.

Contents

1. Scope and delivery status

2. Requirements and acceptance criteria

3. Architecture and deployment boundaries

4. Data model and API contract

5. Extraction and human-review protocol

6. Fixed-test-set evaluation design

7. Authentication, authorization and operations

8. Performance, validation and known limits

9. Runbook and next implementation steps

10. Sources and artifact inventory

1. Scope and delivery status

Parcel serves agents who turn floor plans into reviewable listing data, and clients who explore published properties. An extraction is a machine-produced observation of an uploaded drawing. A review is an independent human revision. An evaluation compares the unmodified extraction with a separately maintained answer key. These are different records and must never overwrite one another.

The delivered hosted prototype provides React screens, Leaflet mapping, Chart.js visualizations, signed-in access, durable uploads, traceable extraction attempts, editable review revisions, publication into the map, and per-run scoring. It contains a fictional San Francisco market dataset and explicitly labelled synthetic demonstration results. These are not MLS data or model performance measurements.

The repository includes a separate FastAPI extraction service with PostgreSQL-compatible persistence, PDF text extraction, bounded rasterization, a configurable vision-model adapter, a frozen synthetic fixture set, a benchmark runner, automated tests, and Docker Compose. The service is runnable but has not been deployed to an external Python host or connected to the live Site. Live vision inference needs credentials and a configured model. No real Gridsnap export or answer key was supplied.

The hosted adapter uses Sites D1 for records and R2 for original bytes. This is an intentional deployment adaptation, not PostgreSQL disguised as another store. PostgreSQL is provided for the separately deployed Python service. PostgreSQL listing and review models are included, but the hosted review/publication flow currently writes D1. Moving that flow to PostgreSQL is an integration step, not a completed capability.

GitHub publication is conditional on a callable authenticated repository connection. The Site source is retained in its managed Git repository; an exportable source ZIP accompanies the handoff when GitHub cannot be reached. A GitHub repository URL must be reported only after actual creation and push succeed.

2. Requirements and acceptance criteria

Interactive map: filter by address/neighborhood, maximum price, bedroom minimum and status; sort listings; select a price marker or list item for details; fit the current result set. Empty searches remain usable and unavailable tiles do not hide listing data.

Property details: show address, price, status, bedrooms, bathrooms, area, price per square foot, agent and reviewed rooms when available. The stock photograph is explicitly illustrative. Fictional listings are visibly distinguished from agent-reviewed records and demonstrations.

Market analytics: compute matching-listing count, median asking price, mean individual price per square foot, mean days on market and bedroom distribution from filtered records. The historical chart is a fixed illustrative series. Real market trends require timestamped, licensed MLS snapshots or transactions; current listing snapshots cannot establish historical sale-price trends.

Floor-plan intake: accept PDF, PNG and JPEG, validate file signatures and size, preserve source bytes and SHA-256, and create a separate run for every requested extraction. The service reads every PDF text layer before considering vision. A complete explicit tagged inventory can be parsed without a model; ordinary labels, incomplete text, scans and photographs require vision.

Review and publication: unknown values remain null. An agent can correct counts, units, area basis, room names, room types and room areas, add/remove rooms, save a new review and publish only after confirming area, bedrooms and bathrooms. A published listing refers to a saved review, not unsaved fields. Publication updates the map and derived listing metrics.

Accuracy: each score refers to an immutable original run and an immutable answer-key hash. Count exact-match accuracy, total-area relative error, room recall, per-room type agreement, per-room area error and extra-room count are separate metrics. No invented overall percentage obscures differing error types.

Security and performance: authentication and ownership checks execute on the server; clients cannot upload or edit. Secrets stay server-side. Upload and rendering limits bound costs. The initial product should remain useful during external map/provider failure. Production throughput, latency and security targets are acceptance targets, not measured claims.

3. Architecture and deployment boundaries

Browser -> Sites application -> D1 and R2. The browser renders React, Leaflet and Chart.js. Same-origin application APIs authorize each operation, store metadata in D1 and stream authorized originals from R2. The browser never receives service or model credentials.

Sites application -> HTTPS FastAPI service -> PostgreSQL and private source volume -> configured vision provider. The gateway uploads the selected source to /v1/extract with a server-only service bearer token and a UUID run ID. FastAPI checks the token, logs the attempt, parses the document, optionally calls a vision-capable model and validates the result. It returns only the structured result and provenance. Its private PostgreSQL log additionally retains the raw successful provider response.

FastAPI is a separate deployment because Python workers are not the Sites JavaScript runtime. Docker Compose starts PostgreSQL and FastAPI for local development. The API binds only to loopback in Compose; deploy behind a TLS reverse proxy or authenticated container host for Sites access. Do not expose the database directly to the Internet.

For the target consolidated architecture, move listings, plans, runs, reviews and evaluations behind FastAPI/PostgreSQL and let Sites serve the frontend and authentication gateway. Preserve the existing layout-v1 JSON contract. Perform a one-time verified migration, switch writes atomically, and avoid indefinite dual-write synchronization. Current D1 and Python logs are explicitly separate records connected by run ID and source hash.

The current service performs bounded synchronous extraction. The gateway times out after 55 seconds and the model request after 45 seconds. For production, replace this with POST job submission, a durable queue, worker leases, status polling, explicit cancellation, a dead-letter queue and reconciliation after ambiguous timeouts. A gateway timeout does not prove the Python task stopped.

4. Data model and API contract

Hosted tables: members(user_id, email, role); plans(id, owner, filename, object_key, sha256, mime, created); runs(id, plan_id, owner, status, version, mode, result, provenance, error, created); reviews(id, run_id, owner, result, created); published(id, review_id, owner, details, created); scores(id, run_id, owner, key_hash, answer_key, metrics, created). Ownership and run-history indexes support actual read patterns. Drizzle generates schema migrations; runtime handlers do not create tables.

Service tables: extraction_runs retain source identity, source path, state, original JSON, provenance, provider response and error; review_revisions, listings and evaluations define the PostgreSQL integration target. The latter tables have models, but their business APIs are not connected to the hosted Site. Production should enforce append-only permissions for completed runs/reviews/evaluations and foreign-key constraints throughout; the prototype enforces ownership and append-only behavior through its API.

The layout-v1 contract requires unit (sqft or sqm), area_basis (internal, gross or unknown), total_area, bedrooms, bathrooms, doors, windows, rooms and warnings. A room has id, name, type and area. Counts and areas may be null, not fabricated zeroes. Room IDs are unique. Bathroom counts support half increments. Areas and counts have finite, nonnegative bounds; total area, when known, must be positive.

GET /api/properties validates minPrice/maxPrice and returns published and sample listings. GET /api/workflow returns the signed-in user's role, service connection state, owned plan metadata, runs, reviews and scores. GET /api/workflow?file=ID returns an original only to its owner. All responses containing user data disable shared caching.

POST /api/workflow?action=upload accepts multipart file bytes. Actions extract and demo create run records; review appends a revision; publish snapshots a saved review into a listing; score compares the original result with a submitted or demonstration key. Cross-origin writes are rejected. Errors preserve unsaved browser fields and uploaded sources.

POST /v1/extract accepts multipart file and run_id under service authentication. Completed duplicate run IDs with matching source return the saved result. Reusing an ID for different bytes or retrying a failed/running ID returns conflict; a deliberate retry uses a new UUID. GET /health checks database connectivity and reports whether vision configuration exists without revealing credentials.

5. Extraction and human-review protocol

First verify the signature, maximum 10 MB input size and supported format. Reject encrypted PDFs, more than 10 pages, and images exceeding 20 megapixels. Original bytes are retained under opaque IDs. Filenames are metadata, never filesystem paths. The service strips image metadata while preparing model input and limits the longest raster edge to 2048 pixels.

For a PDF, extract text from every page. The tagged fast path accepts only a complete declared inventory: UNIT, AREA_BASIS, TOTAL_AREA, BEDROOMS, BATHROOMS, DOORS, WINDOWS and ROOM records. It is a narrow parser for explicit labels, not a geometric detector. The synthetic tagged fixture exercises it. Gridsnap's own JSON/export schema is not assumed to match this syntax.

If that inventory is incomplete, send rendered pages plus extracted text to the configured vision model. A PNG/JPEG enters the same vision path directly. The prompt treats document content as untrusted data, preserves visible room IDs, specifies opening-count conventions, distinguishes internal/gross area and asks for null where dimensions or evidence are insufficient. Strict JSON-schema output is requested, then Pydantic validates the response independently. A provider refusal, truncation or invalid layout fails the run.

The user must review results against the source. Do not silently sum room areas into gross area: halls, walls, stairs, voids and exclusions change the definition. For bare drawings without labels or scale, absolute area and room function may be unidentifiable. Abstaining is preferable to manufactured precision; evaluate abstention coverage separately from correctness.

A successful extraction records source/text/prompt hashes, schema and prompt versions, requested/returned model, preprocessing, provider response ID, system fingerprint when supplied, token usage, timing and service revision. Defaults and unsupported seeds are recorded honestly. This permits traceability and controlled reruns; it does not guarantee bit-identical responses from a hosted model.

Original results remain unchanged after review. Saving appends a review ID. Publishing checks mandatory fields and snapshots that exact revision. Scores always read runs.result. The current UI supports manual review after a failed extraction but keeps the failure and its missing machine output intact; such a review is not a successful automated extraction.

6. Fixed-test-set evaluation design

Obtain actual Gridsnap source exports, renderings and independently verified truth. Public documentation establishes PDF/PNG and project-file export, but does not establish that every export includes an evaluation-ready answer key. Verify the supplied version and schema before writing an adapter. Keep a normalized key beside each exported drawing and record the project version, exporter settings, floor identity and measurement conventions.

Use three tiers: tagged drawings with explicit IDs and measurements; labelled drawings with names/dimensions but no machine-readable inventory; bare drawings with walls/openings and progressively reduced annotations. Decide beforehand which fields are observable in each tier. Do not strip a label and still treat the withheld function as recoverable from geometry alone. Count unknowns in coverage and report an observable-field score separately when truth contains unobservable fields.

Partition by source building before creating variants. All floors, image/PDF variants and annotation tiers of one building remain in the same partition to prevent leakage. Use development cases for prompts and rendering choices; freeze the held-out test manifest, all file/key hashes and metric version before comparing extraction versions. Do not send answer keys, expected counts or benchmark metadata to the model.

The included manifest is parcel-synthetic-smoke-v1: one schematic layout in three tiers. It is a software smoke test, not statistical evidence about floor-plan extraction. The drawings are not to scale; labelled internal areas are declared fixture facts. The bare case intentionally lacks enough information to infer those facts. All three variants are one source group and cannot support generalization estimates.

The evaluator verifies source and key hashes, sends only source bytes, loads truth for scoring after the request, saves per-case provenance, includes failed cases and reports coverage plus failure-inclusive count accuracy. Runs persist independently in the Python database. The JSON report is the aggregate evaluation artifact. Statistical confidence intervals, multi-run variance, room correspondence for untagged drawings and dataset-level database ingestion are future evaluation work, not delivered measurements.

Count accuracy = exact matching known key counts divided by the number of known key counts. Unknown predicted values count as errors when truth is known. Total-area relative error = absolute(predicted - true)/true after unit conversion, only with matching, known area basis. Room recall = matched stable IDs / expected IDs; extra predicted rooms are reported separately. Per-room type and area errors are retained, but ID matching alone cannot establish geometric correctness.

For bare drawings, add a fixed geometry-based correspondence method (for example polygon overlap followed by bipartite matching) before claiming room-level model accuracy. Do not let a reviewer inspect truth and then remap predictions ad hoc. Maintain that matcher as a separately versioned metric. Report results per tier, field and model version, with sample size, failures, latency, cost and confidence intervals.

7. Authentication, authorization and operations

The Site starts owner-private and uses platform sign-in. Server handlers require the authenticated user identity. The first identity in this owner-private deployment becomes the bootstrap agent; later identities default to client. This bootstrap relies on owner-private deployment. Before widening access, replace it with an explicit administrator/agent allowlist or managed membership process. The UI role label is informational; it cannot grant privileges.

Agents can read their own original plans, runs, reviews and scores and create reviewed listings. Clients can read published listing details and market summaries only. Raw plan bytes and extraction histories are not returned to clients. Multi-agent organizations, delegated review, agent invitations and role management are outside the current UI. Sharing the Site and granting an application role are separate controls.

Only the gateway holds EXTRACTION_SERVICE_TOKEN. Only Python holds VISION_API_KEY. Store secrets in deployment settings, never Git, browser bundles, fixture metadata or logs. Rotate the service token on suspected exposure. The service is an internal trusted caller boundary, not an end-user authentication provider. Production needs a private service network or rate-limited ingress in addition to bearer authentication.

Retention should be chosen with the listing owner before operational use. Production requirements include encrypted volumes/backups, least-privilege database credentials, original-plan deletion and backup-expiry policies, malware/parser isolation, dependency scanning, request-rate quotas, model-spend ceilings and provider data-processing terms. The current prototype does not claim regulatory certification or hardened multi-tenant readiness.

Operate each worker with memory/CPU limits. Track attempt state, extraction failure rate, parse failures, queue delay (when queues exist), latency by tier, model cost, review correction rates and publication failures. Never log authorization headers or full source plans to generic console logs. Reconcile stale running rows after process termination; do not simply relabel them successful.

For production PostgreSQL use versioned migrations and tested rollback/restore plans. DB_AUTO_CREATE is a development bootstrap only. Back up PostgreSQL and original files consistently, retaining source hashes and references. Validate restoration by retrieving a known source and rerunning its score, not just by checking that a backup file exists.

8. Performance, validation and known limits

Leaflet and Chart.js load on demand, map markers are regenerated from the current result set, and map size follows responsive layout changes. The current API caps saved listings at 200 and history at 100. These are MVP bounds, not a scalable search architecture. Add cursor pagination, viewport/bounding-box queries, spatial indexes, server-side aggregation and marker clustering before supporting large portfolios.

Proposed performance gates: p95 listing/filter API under 300 ms at the chosen regional deployment; map interactions at 30 fps or better with the agreed marker budget; extraction timeout/failure rate reported independently of successful latency; 100 percent of accepted extraction attempts traceable by run ID. Measure on specified hardware and concurrent load. None of these targets is presented as a measured result here.

Validation completed for this handoff: TypeScript compile and production build; twelve Python tests covering tagged PDF parsing, incomplete-text fallback, unsupported inputs, missing vision configuration, service authentication, idempotency/conflicting IDs, failed-run persistence, duplicate room IDs, unit conversion, area-basis mismatch, missing-room recall and unknown-count penalties. Python tests use a temporary SQLite database to exercise persistence logic. PostgreSQL container execution, real model calls, hosted role testing with two users, and Gridsnap benchmark execution have not been performed.

Browser automation was unavailable under the permitted preview workflow. The code is responsive and accessible primitives support focus/keyboard behavior, but visual and assistive-technology QA still needs a browser pass. Optional WebMCP filtering is feature-detected and shares visible filter state; supported-browser registration tests were unavailable and this feature is not a delivery gate.

Known limitations: live MLS is absent; the trend series is illustrative; Python is not hosted by this delivery; no vision credential/model is configured; actual Gridsnap fixtures are absent; the current Python tagged parser is intentionally narrow; the hosted and Python stores are not unified; current room matching requires stable IDs; process crashes can leave running rows until reconciliation; multi-tenant hardening and performance benchmarks remain work. These are visible boundaries, not hidden simulated successes.

9. Runbook and next implementation steps

Hosted walkthrough: sign in, open Floor-plan studio, create a demonstration run, inspect the source PDF, score against the fixed key, correct the deliberate errors, save the review, enter a listing title/price/location and publish. Return to Market explorer to find the reviewed demonstration listing. Its source remains labelled Demonstration. A normal upload saves privately and an unavailable extractor produces a failed, retained attempt.

Local Python: use Python 3.12; create a virtual environment; install extraction-service/requirements.txt. Set DATABASE_URL, DB_AUTO_CREATE=1 for local development, UPLOAD_DIR and a strong SERVICE_TOKEN. Start uvicorn app.main:app from extraction-service. Alternatively set POSTGRES_PASSWORD and SERVICE_TOKEN and run docker compose up --build from the repository root. Compose keeps its published port on localhost.

Run python -m unittest discover -s tests -v from extraction-service. To run the fixed smoke set against the local service, set SERVICE_TOKEN and run python evaluate.py fixtures/manifest.json --output evaluation-report.json. Without vision settings, the tagged case succeeds and the other tiers correctly fail; this is not a model benchmark. Enable a supported vision-capable model and credentials before measuring inference.

Connect a deployed HTTPS service through Sites secrets EXTRACTION_SERVICE_URL and EXTRACTION_SERVICE_TOKEN. Configure VISION_API_KEY, VISION_MODEL, DATABASE_URL and a commit-based SERVICE_REVISION on the Python host. Use a small tagged PDF for the first end-to-end connectivity check, then a labelled plan to confirm the real model path. Verify source and run IDs agree in both logs.

Next priority is supplying a representative Gridsnap export and its independent answer key. Confirm the schema and unit/area/opening conventions, implement the adapter, and freeze a building-level split. After that, connect the model service, test two real roles, and run a held-out baseline before choosing an accuracy target. Consolidate PostgreSQL business APIs before treating this prototype as the final production architecture.

10. Sources and artifact inventory

Primary documentation consulted: Leaflet reference (https://leafletjs.com/reference); Chart.js line charts and performance guidance (https://www.chartjs.org/docs/latest/charts/line.html and https://www.chartjs.org/docs/latest/general/performance.html); PyMuPDF page/text/raster APIs (https://pymupdf.readthedocs.io/en/latest/page.html); Gridsnap features and PDF export (https://gridsnap.app/features/ and https://gridsnap.app/help/export-pdf/); OpenAI structured outputs and vision formats (https://developers.openai.com/api/docs/guides/structured-outputs and https://developers.openai.com/api/docs/guides/images-vision). Provider/model availability must be verified for the deployment account.

Main source: app/dashboard.tsx, app/floor-plans.tsx, app/property-map.tsx, app/chart.tsx; API handlers under app/api; JSON validation/scoring in lib/layout-data.ts; ownership and storage helpers in lib/server.ts; schema and migrations in db and drizzle; Python code in extraction-service/app; fixtures and fixed manifest in extraction-service/fixtures; unit/integration tests in extraction-service/tests; benchmark runner extraction-service/evaluate.py; deployment recipe compose.yaml; document source docs/software-design.md; rendered document public/software-design.pdf.

Neighborhood photo: Waterthedog, Waller Street Painted Ladies, San Francisco, Wikimedia Commons, released to the public domain. Source: https://commons.wikimedia.org/wiki/File:Waller_Street_Painted_Ladies,_San_Francisco.jpg. It is illustrative and is not associated with a real listing. OpenStreetMap contributors are credited directly on the map.